

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Deep Learning Assignment Report

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.

1 Load the dataset.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import MinMaxScaler
7 from sklearn.metrics import accuracy_score, precision_score,
8   recall_score
9 from sklearn.metrics import f1_score, confusion_matrix,
10   ConfusionMatrixDisplay
```

- Display the first five samples.

2 Display the first five samples.

2.1 Python Code

```
1 columns = ["Variance", "Skewness", "Curtosis", "Entropy", "Class"]
2
3 df = pd.read_csv("data_banknote_authentication.txt",
4                 names=columns)
5
6 print(df.head())
```

2.2 Output

	Variance	Skewness	Curtosis	Entropy	Class
0	3.62160	8.6661	-2.8073	-0.44699	0
1	4.54590	8.1674	-2.4586	-1.46210	0
2	3.86600	-2.6383	1.9242	0.10645	0
3	3.45660	9.5228	-4.0112	-3.59440	0
4	0.32924	-4.4552	4.5718	-0.98880	0

- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

3 Determine dataset dimensions, Identify missing values, Display descriptive statistics.

3.1 Python Code

```
1 print("Shape:", df.shape)
2
3 print("\nMissing Values")
4 print(df.isnull().sum())
5
6 print("\nDataset Info")
7 print(df.info())
8
9 print("\nStatistics")
10 print(df.describe())
```

3.2 Output

Shape: (1372, 5)

Missing Values

Variance 0
Skewness 0
Curtosis 0
Entropy 0
Class 0
dtype: int64

Dataset Info

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1372 entries, 0 to 1371

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	Variance	1372 non-null	float64
1	Skewness	1372 non-null	float64
2	Curtosis	1372 non-null	float64
3	Entropy	1372 non-null	float64
4	Class	1372 non-null	int64

dtypes: float64(4), int64(1)

memory usage: 53.7 KB

None

Statistics

	Variance	Skewness	Curtosis	Entropy	Class
count	1372.000000	1372.000000	1372.000000	1372.000000	1372.000000
mean	0.433735	1.922353	1.397627	-1.191657	0.444606

std	2.842763	5.869047	4.310030	2.101013	0.497103
min	-7.042100	-13.773100	-5.286100	-8.548200	0.000000
25%	-1.773000	-1.708200	-1.574975	-2.413450	0.000000
50%	0.496180	2.319650	0.616630	-0.586650	0.000000
75%	2.821475	6.814625	3.179250	0.394810	1.000000
max	6.824800	12.951600	17.927400	2.449500	1.000000

Task 2: Exploratory Data Analysis

Generate Histogram

4 Histograms

4.1 Python Code

```
1 df.hist(figsize=(10,8))
2 plt.tight_layout()
3 plt.show()
```

Output

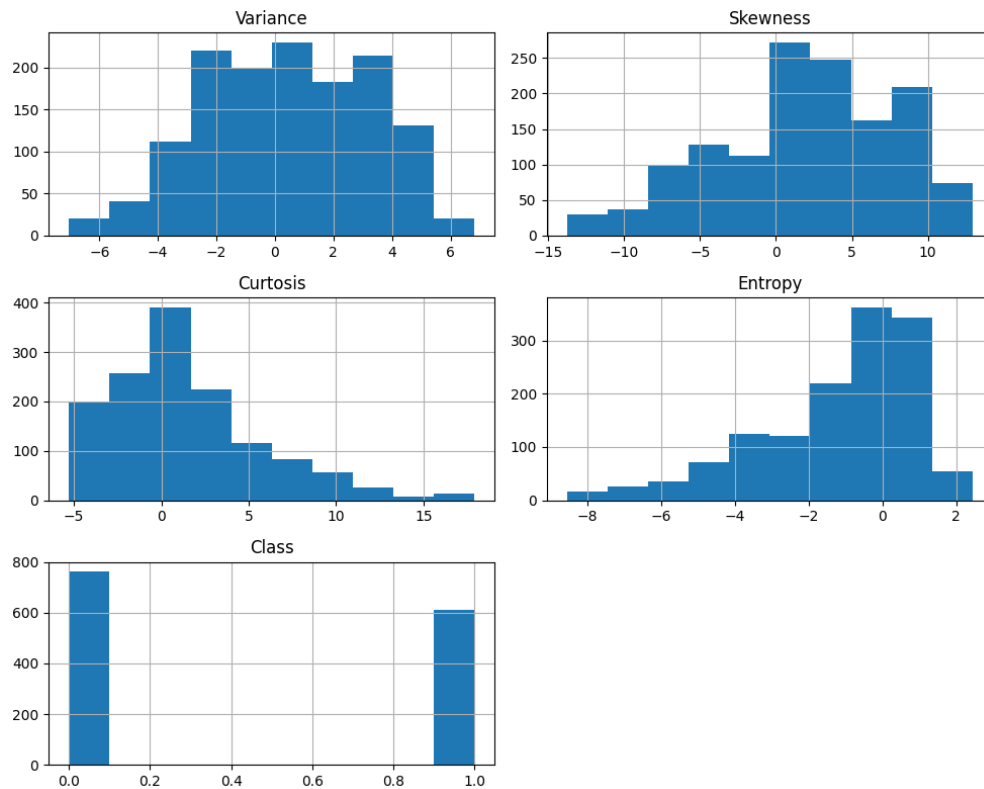


Figure 1: Histogram of the Dataset

Inference: All features are skewed and spread out – no neat bell curves. Multiple peaks suggest the two classes are naturally separated.

Correlation Heatmap

5 Correlation Heatmap

5.1 Python Code

```
1 plt.figure(figsize=(8,6))
2 sns.heatmap(df.corr(),
3             annot=True,
4             cmap="coolwarm")
5
6 plt.title("Correlation Heatmap")
7 plt.show()
```

Output

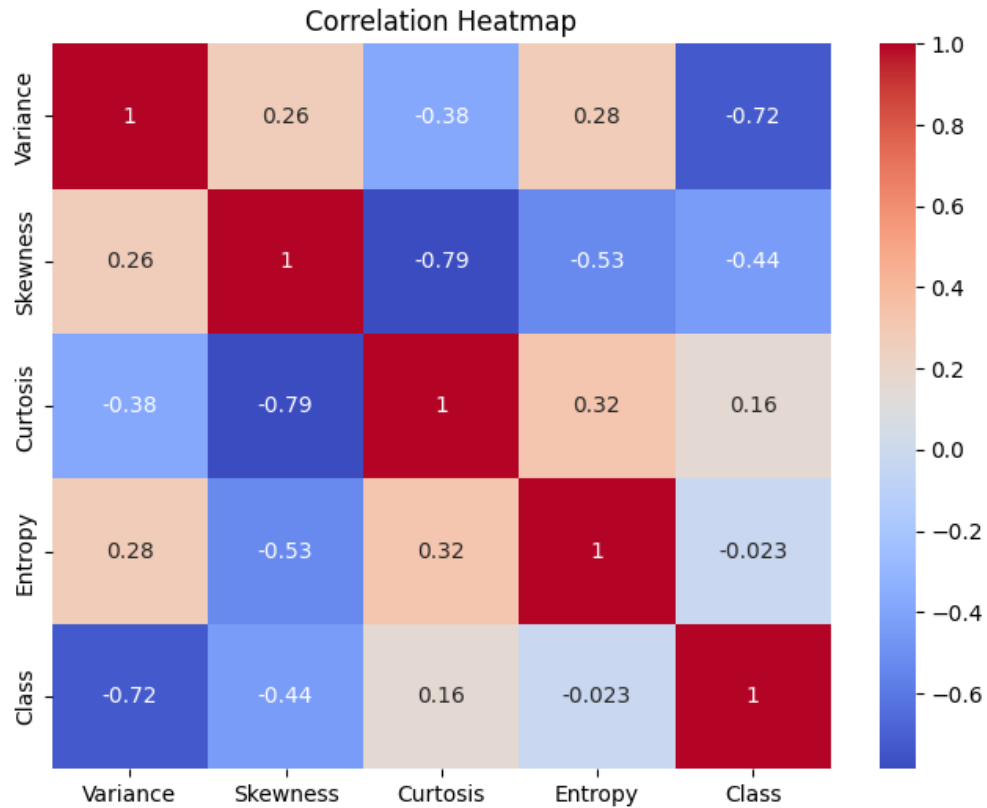


Figure 2: Histogram of the Dataset

Inference: No strong correlations between features – they're mostly independent. No redundant info to mess up the perceptron. Scatter Plot

6 Scatter Plot

6.1 Python Code

```
1 plt.figure(figsize=(7,6))
2
3 sns.scatterplot(
4     x="Variance",
5     y="Skewness",
6     hue="Class",
7     data=df
8 )
9
10 plt.show()
```

6.2 Output

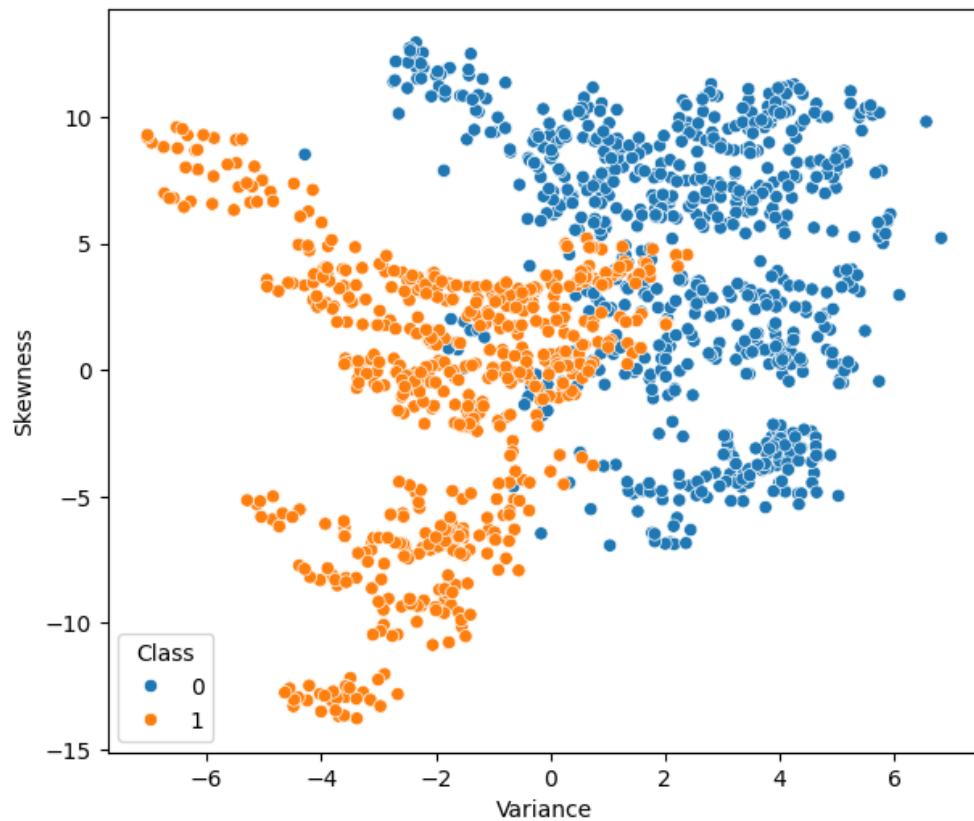


Figure 3: Histogram of the Dataset

Inference: The two classes form clear, separate clusters. They're roughly linearly separable – exactly why our perceptron works well.

7 Box Plot

7.1 Python Code

```

1 plt.figure(figsize=(10,6))
2
3 for i,col in enumerate(df.columns[:-1]):
4     plt.subplot(2,2,i+1)
5     sns.boxplot(y=df[col])
6
7 plt.tight_layout()
8 plt.show()

```

7.2 Output

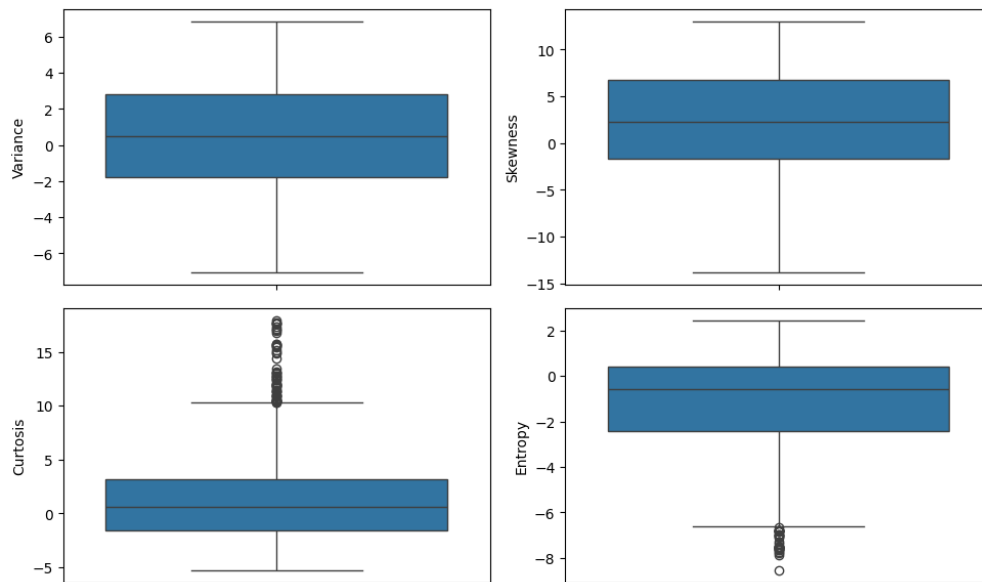


Figure 4: Histogram of the Dataset

Inference: Features have very different scales and quite a few outliers. Normalisation was a must – otherwise big numbers dominate weight updates.

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

8 Normalizing all numerical features and Split Data

8.1 Python Code

```
1 X = df.iloc[:, :-1].values
2 y = df.iloc[:, -1].values
3
4 scaler = MinMaxScaler()
5
6 X = scaler.fit_transform(X)
7
8 X_train, X_test, y_train, y_test = train_test_split(
9     X,
10    y,
11    test_size=0.2,
12    random_state=42
13 )
```

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

9 Perceptron

9.1 Python Code

```
1 import numpy as np
2
3 class Perceptron:
4
5     def __init__(self, lr=0.01, epochs=20):
6         self.lr = lr
7         self.epochs = epochs
8         self.weights = None
9         self.bias = 0
10
11     def activation(self, z):
12         return np.where(z >= 0, 1, 0)
13
14     def fit(self, X, y):
15
```

```

16         n_features = X.shape[1]
17
18         self.weights = np.zeros(n_features)
19         self.bias = 0
20
21         self.errors = []
22         self.weight_history = []
23         self.bias_history = []
24
25         for epoch in range(self.epochs):
26
27             errors = 0
28
29             for xi, target in zip(X, y):
30
31                 z = np.dot(xi, self.weights) + self.bias
32                 prediction = self.activation(z)
33
34                 update = self.lr * (target - prediction)
35
36                 self.weights += update * xi
37                 self.bias += update
38
39                 if update != 0:
40                     errors += 1
41
42                 self.errors.append(errors)
43                 self.weight_history.append(self.weights.copy())
44                 self.bias_history.append(self.bias)
45
46                 print(f"Epoch {epoch+1}: Errors = {errors}")
47
48     def predict(self, X):
49         z = np.dot(X, self.weights) + self.bias
50         return self.activation(z)

```

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

10 Train Model

10.1 Python Code

```
1 model = Perceptron(lr=0.01, epochs=20)
2 model.fit(X_train, y_train)
3
4 y_pred = model.predict(X_test)
```

10.2 Output

```
Epoch 1: Errors = 166
Epoch 2: Errors = 88
Epoch 3: Errors = 57
Epoch 4: Errors = 51
Epoch 5: Errors = 47
Epoch 6: Errors = 45
Epoch 7: Errors = 48
Epoch 8: Errors = 32
Epoch 9: Errors = 19
Epoch 10: Errors = 40
Epoch 11: Errors = 35
Epoch 12: Errors = 28
Epoch 13: Errors = 40
Epoch 14: Errors = 31
Epoch 15: Errors = 28
Epoch 16: Errors = 38
Epoch 17: Errors = 27
Epoch 18: Errors = 28
Epoch 19: Errors = 32
Epoch 20: Errors = 31
```

Inference: Errors dropped from 166 to around 30 in no time. Minor wobbles later – typical when data isn't perfectly separable.

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score

11 Modal Evaluation

11.1 Python Code

```
1 y_pred=model.predict(X_test)
2
3 print("Accuracy :",accuracy_score(y_test,y_pred))
4
5 print("Precision :",precision_score(y_test,y_pred))
6
7 print("Recall :",recall_score(y_test,y_pred))
8
9 print("F1 Score :",f1_score(y_test,y_pred))
```

11.2 Output

Accuracy : 0.9745454545454545
Precision : 0.9477611940298507
Recall : 1.0
F1 Score : 0.9731800766283525

- Confusion Matrix

12 Confusion Matrix

12.1 Python Code

```
1 cm=confusion_matrix(y_test,y_pred)
2
3 disp=ConfusionMatrixDisplay(cm)
4
5 disp.plot()
6
7 plt.show()
```

Output

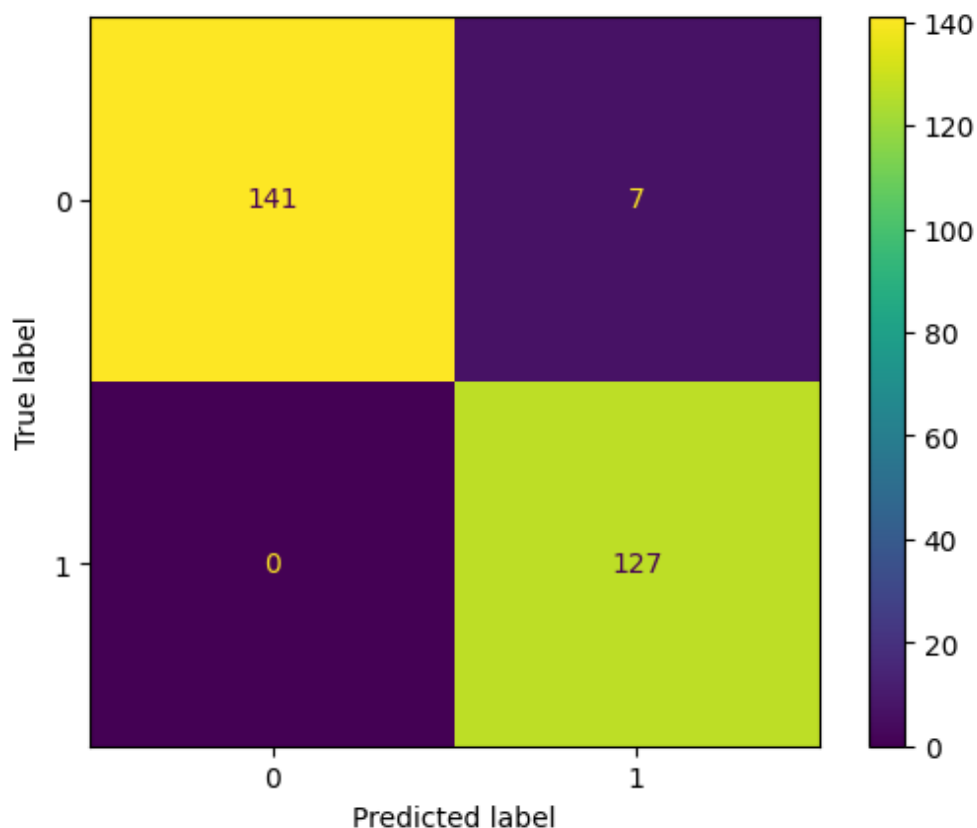


Figure 5: Histogram of the Dataset

Inference: Almost everything lies on the diagonal – very few misclassifications. The perceptron clearly nailed the real vs fake distinction.

7. Mandatory Plots

Plot	Purpose	Expected Inference
Feature Histograms	Distribution Analysis	Identify skewness and outliers.
Correlation Heatmap	Feature Relationship	Observe highly correlated features.
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable.
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron.
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights.

Bias Evolution	Parameter Analysis	Understand the role of the bias term.
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN.
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates.
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features.

Students should provide a brief interpretation (2-3 sentences) for every generated plot.

13 Weight Evolution

13.1 Python Code

```

1 weights=np.array(model.weight_history)
2
3 for i in range(weights.shape[1]):
4     plt.plot(weights[:,i],label=f"W{i+1}")
5
6 plt.legend()
7
8 plt.xlabel("Epoch")
9
10 plt.ylabel("Weight Value")
11
12 plt.title("Weight Evolution")
13
14 plt.grid()
15
16 plt.show()

```

Output

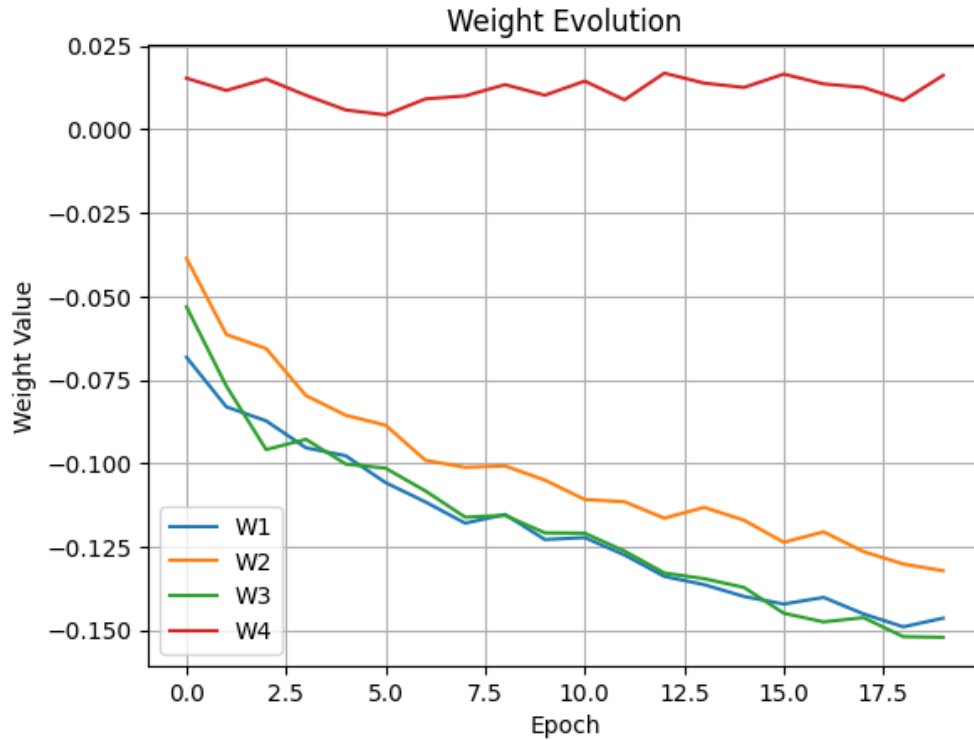


Figure 6: Histogram of the Dataset

Inference: Weights settle quickly after the first few epochs and stay steady. No wild oscillations – training was smooth and converged well.

Output

14 Bias Evolution

14.1 Python Code

```

1 plt.plot(model.bias_history,
2           marker='o')
3
4 plt.xlabel("Epoch")
5
6 plt.ylabel("Bias")
7
8 plt.title("Bias Evolution")
9
10 plt.grid()
11
12 plt.show()

```

14.2 Output

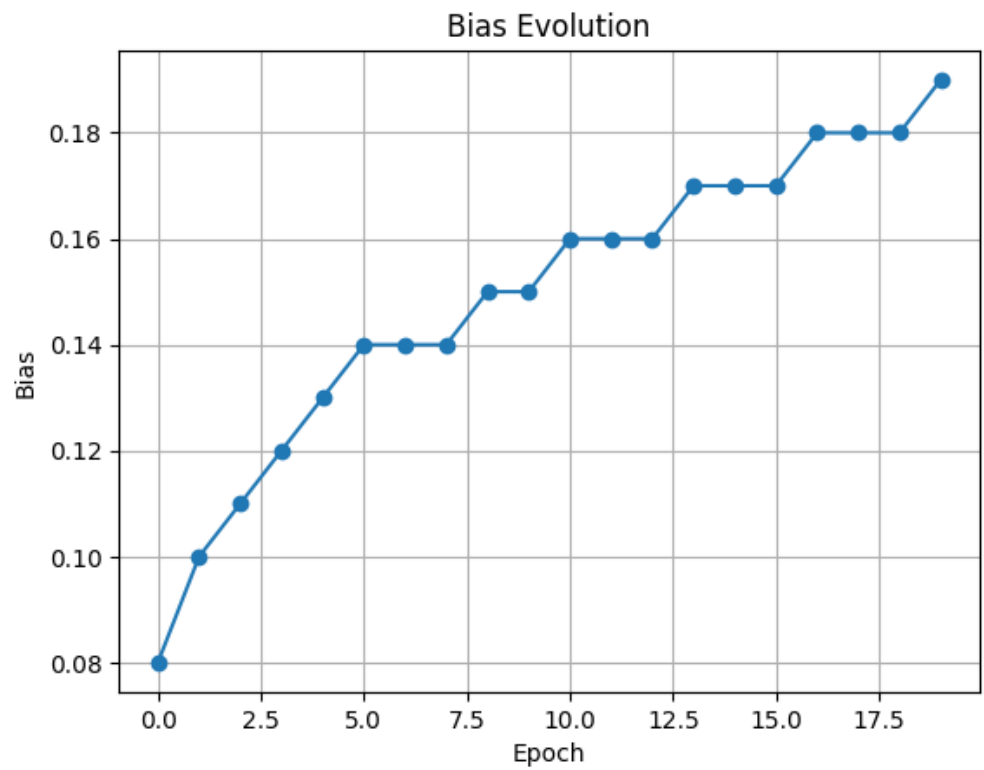


Figure 7: Histogram of the Dataset

Inference: Bias jumps sharply at the start, then plateaus. It just shifts the decision boundary to the right place and stays put.

8. Performance Tables

Training Summary

	Parameter	Value
0	Dataset Size	1372
1	Train/Test Split	1097 / 275
2	Learning Rate	0.01
3	Epochs	20
4	Final Weights	$[-0.1464, -0.1322, -0.1521, 0.0161]$
5	Final Bias	0.19
6	Accuracy	0.9745
7	Precision	0.9478
8	Recall	1.0
9	F1-score	0.9732

Epoch-wise Learning

9. Additional Tasks

1. Compare the Step and Sigmoid activation functions. Discuss why the Step Activation Function is not used in modern deep learning models.
2. Compare the implementation with Scikit-learn's Perceptron.
3. Repeat the experiment using learning rates 0.001, 0.01 and 0.1.

15 Experiment Using Different Learning Rates

15.1 Python Code

```
1 rates=[0.001,0.01,0.1]
2
3 plt.figure(figsize=(8,6))
4
5 for lr in rates:
6
7     p=Perceptron(
8         lr=lr,
9         epochs=20
10    )
11
12    p.fit(X_train,y_train)
13
14    plt.plot(p.errors,label=f"LR={lr}")
15
16 plt.legend()
17
18 plt.xlabel("Epoch")
19
20 plt.ylabel("Errors")
21
22 plt.title("Learning Rate Comparison")
23
24 plt.grid()
25
26 plt.show()
```

15.2 Output

```
Epoch 1: Errors = 166
Epoch 2: Errors = 88
Epoch 3: Errors = 57
Epoch 4: Errors = 51
Epoch 5: Errors = 47
Epoch 6: Errors = 45
Epoch 7: Errors = 48
```

Epoch 8: Errors = 32
Epoch 9: Errors = 19
Epoch 10: Errors = 40
Epoch 11: Errors = 35
Epoch 12: Errors = 28
Epoch 13: Errors = 40
Epoch 14: Errors = 31
Epoch 15: Errors = 28
Epoch 16: Errors = 38
Epoch 17: Errors = 27
Epoch 18: Errors = 28
Epoch 19: Errors = 32
Epoch 20: Errors = 31
Epoch 1: Errors = 166
Epoch 2: Errors = 88
Epoch 3: Errors = 57
Epoch 4: Errors = 51
Epoch 5: Errors = 47
Epoch 6: Errors = 45
Epoch 7: Errors = 48
Epoch 8: Errors = 32
Epoch 9: Errors = 19
Epoch 10: Errors = 40
Epoch 11: Errors = 35
Epoch 12: Errors = 28
Epoch 13: Errors = 40
Epoch 14: Errors = 31
Epoch 15: Errors = 28
Epoch 16: Errors = 38
Epoch 17: Errors = 27
Epoch 18: Errors = 28
Epoch 19: Errors = 32
Epoch 20: Errors = 31
Epoch 1: Errors = 166
Epoch 2: Errors = 88
Epoch 3: Errors = 57
Epoch 4: Errors = 51
Epoch 5: Errors = 47
Epoch 6: Errors = 45
Epoch 7: Errors = 48
Epoch 8: Errors = 32
Epoch 9: Errors = 19
Epoch 10: Errors = 40
Epoch 11: Errors = 35
Epoch 12: Errors = 28
Epoch 13: Errors = 40
Epoch 14: Errors = 31
Epoch 15: Errors = 28

Epoch 16: Errors = 38
Epoch 17: Errors = 27
Epoch 18: Errors = 28
Epoch 19: Errors = 32
Epoch 20: Errors = 31

Inference: Errors dropped from 166 to around 30 in no time. Minor wobbles later – typical when data isn't perfectly separable.

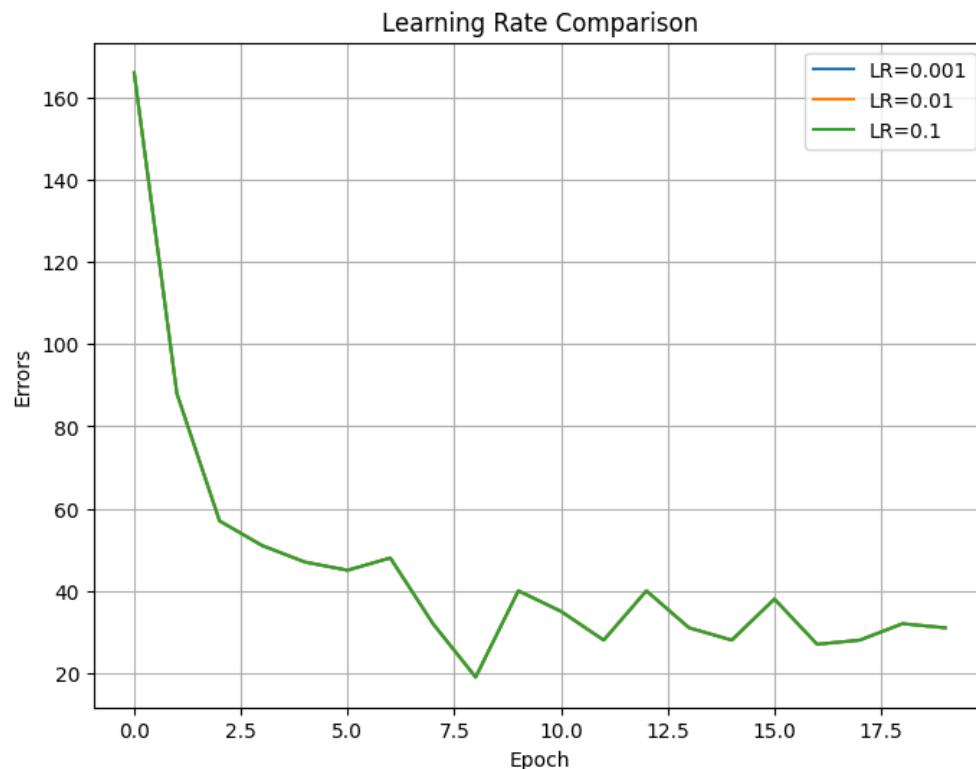


Figure 8: Histogram of the Dataset

Inference: LR = 0.01 is the sweet spot – fast enough to learn, stable enough not to bounce. 0.1 is jumpy, 0.001 is painfully slow. Middle ground wins.

4. Explain why a Single Layer Perceptron cannot solve the XOR problem.
5. Study the effect of feature normalization on model convergence.

10. Report Contents

The report should include:

- Objective

- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code
- Experimental Results
- Generated Plots with Interpretation
- Performance Tables
- Discussion
- Conclusion
- References

11. References

1. F. Rosenblatt, “The Perceptron,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.

Additional Tasks

1. Code the perceptron learning algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages. (K4)
2. Code the perceptron learning algorithm for the XOR gate using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python’s built-in packages. Analyze and identify the pattern that prevents the algorithm from converging. (K4)

16 Task-1

16.1 Python Code

```

1 class Perceptron:
2
3     def __init__(self, lr=0.1, epochs=20):
4         self.lr = lr
5         self.epochs = epochs
6         self.weights = None
7         self.bias = 0
8
9     def activation(self, x):
10        return 1 if x >= 0 else 0
11
12    def predict(self, X):
13        linear = np.dot(X, self.weights) + self.bias
14        return np.array([self.activation(i) for i in linear])
15
16    def plot_boundary(self, X, y, title):
17
18        # ----- NOT Gate -----
19        if X.shape[1] == 1:
20
21            plt.figure(figsize=(5,2))
22
23            colors = ['red' if i==0 else 'blue' for i in y]
24
25            plt.scatter(X[:,0], np.zeros(len(X)),
26                        c=colors, s=120)
27
28            if abs(self.weights[0]) > 1e-8:
29                threshold = -self.bias/self.weights[0]
30                plt.axvline(threshold,
31                            color='black',
32                            label='Decision Boundary')
33
34            plt.yticks([])
35            plt.xlim(-0.5,1.5)
36            plt.grid(True)
37            plt.title(title)
38            plt.legend()
39            plt.show()
40
41            return
42
43        # ----- AND / OR / XOR -----
44
45        plt.figure(figsize=(5,5))
46
47        colors = ['red' if i==0 else 'blue' for i in y]
48
49        plt.scatter(X[:,0], X[:,1],
50                    c=colors,
51                    s=120)
52
53        if abs(self.weights[1]) > 1e-8:

```

```

54
55     x = np.linspace(-0.5,1.5,100)
56
57     y_line = -(self.weights[0]*x+self.bias)/self.weights[1]
58
59     plt.plot(x,y_line,
60              'k-',
61              label='Decision Boundary')
62
63     plt.xlim(-0.5,1.5)
64     plt.ylim(-0.5,1.5)
65     plt.grid(True)
66     plt.title(title)
67     plt.legend()
68     plt.show()
69
70     def fit(self, X, y, gate_name):
71
72         self.weights = np.zeros(X.shape[1])
73         self.bias = 0
74
75         update = 1
76
77         print("\n=====")
78         print(f"Training Perceptron for {gate_name}")
79         print("=====")
80
81         for epoch in range(self.epochs):
82
83             error_count = 0
84
85             for xi, target in zip(X, y):
86
87                 linear = np.dot(xi, self.weights) + self.bias
88
89                 pred = self.activation(linear)
90
91                 error = target - pred
92
93                 if error != 0:
94
95                     self.weights += self.lr * error * xi
96                     self.bias += self.lr * error
97
98                     print(f"\nUpdate {update}")
99                     print("Input :", xi)
100                    print("Target :", target)
101                    print("Prediction :", pred)
102                    print("Weights :", self.weights)
103                    print("Bias :", self.bias)
104
105                    # Plot every update except limit XOR to first 10
106                    if gate_name == "XOR":
107                        if update <= 10:

```

```

108         self.plot_boundary(
109             X,
110             y,
111             f"{gate_name} Update {update}"
112         )
113     else:
114         self.plot_boundary(
115             X,
116             y,
117             f"{gate_name} Update {update}"
118         )
119
120         update += 1
121         error_count += 1
122
123     if error_count == 0:
124         print("\nConverged Successfully!")
125         break
126
127     print("\nFinal Weights :", self.weights)
128     print("Final Bias :", self.bias)
129     print("Predictions :", self.predict(X))

```

16.2 AND

```

1 X = np.array([
2     [0,0],
3     [0,1],
4     [1,0],
5     [1,1]
6 ])
7
8 y = np.array([0,0,0,1])
9
10 model = Perceptron(lr=0.1, epochs=20)
11 model.fit(X, y, "AND")

```

```

Update 1
Input : [0 0]
Target : 0
Prediction : 1
Weights : [0. 0.]
Bias : -0.1

```

```

Update 2
Input : [1 1]
Target : 1
Prediction : 0
Weights : [0.1 0.1]
Bias : 0.0

```

Update 3
Input : [0 0]
Target : 0
Prediction : 1
Weights : [0.1 0.1]
Bias : -0.1

Update 4
Input : [0 1]
Target : 0
Prediction : 1
Weights : [0.1 0.]
Bias : -0.2

Update 5
Input : [1 1]
Target : 1
Prediction : 0
Weights : [0.2 0.1]
Bias : -0.1

Update 6
Input : [0 1]
Target : 0
Prediction : 1
Weights : [0.2 0.]
Bias : -0.2

Update 7
Input : [1 0]
Target : 0
Prediction : 1
Weights : [0.1 0.]
Bias : -0.30000000000000004

Update 8
Input : [1 1]
Target : 1
Prediction : 0
Weights : [0.2 0.1]
Bias : -0.20000000000000004

Converged Successfully!

Final Weights : [0.2 0.1]
Final Bias : -0.20000000000000004
Predictions : [0 0 0 1]

16.3 OR

```
1 X = np.array([
2     [0,0],
3     [0,1],
4     [1,0],
5     [1,1]
6 ])
7
8 y = np.array([0,1,1,1])
9
10 model = Perceptron(lr=0.1, epochs=20)
11 model.fit(X, y, "OR")
```

Update 1

Input : [0 0]

Target : 0

Prediction : 1

Weights : [0. 0.]

Bias : -0.1

Update 2

Input : [0 1]

Target : 1

Prediction : 0

Weights : [0. 0.1]

Bias : 0.0

Update 3

Input : [0 0]

Target : 0

Prediction : 1

Weights : [0. 0.1]

Bias : -0.1

Update 4

Input : [1 0]

Target : 1

Prediction : 0

Weights : [0.1 0.1]

Bias : 0.0

Update 5

Input : [0 0]

Target : 0

Prediction : 1
Weights : [0.1 0.1]
Bias : -0.1

Converged Successfully!

Final Weights : [0.1 0.1]
Final Bias : -0.1
Predictions : [0 1 1 1]

16.4 NOR

```
1 X = np.array([
2     [0],
3     [1]
4 ])
5
6 y = np.array([1,0])
7
8 model = Perceptron(lr=0.1, epochs=20)
9 model.fit(X, y, "NOT")
```

Update 1
Input : [1]
Target : 0
Prediction : 1
Weights : [-0.1]
Bias : -0.1

Update 2
Input : [0]
Target : 1
Prediction : 0
Weights : [-0.1]
Bias : 0.0

Converged Successfully!

Final Weights : [-0.1]
Final Bias : 0.0
Predictions : [1 0]

17 Task-2

17.1 XOR

```
1 X = np.array([
2     [0,0],
3     [0,1],
4     [1,0],
5     [1,1]
6 ])
7
8 y = np.array([0,1,1,0])
9
10 model = Perceptron(lr=0.1, epochs=20)
11 model.fit(X, y, "XOR")
```

Update 1
Input : [0 0]
Target : 0
Prediction : 1
Weights : [0. 0.]
Bias : -0.1

Update 2
Input : [0 1]
Target : 1
Prediction : 0
Weights : [0. 0.1]
Bias : 0.0

Update 3
Input : [1 1]
Target : 0
Prediction : 1
Weights : [-0.1 0.]
Bias : -0.1

Update 4
Input : [0 1]
Target : 1
Prediction : 0
Weights : [-0.1 0.1]
Bias : 0.0

Update 5

Input : [1 0]
Target : 1
Prediction : 0
Weights : [0. 0.1]
Bias : 0.1

Update 6
Input : [1 1]
Target : 0
Prediction : 1
Weights : [-0.1 0.]
Bias : 0.0

Update 7
Input : [0 0]
Target : 0
Prediction : 1
Weights : [-0.1 0.]
Bias : -0.1

Update 8
Input : [0 1]
Target : 1
Prediction : 0
Weights : [-0.1 0.1]
Bias : 0.0

Update 9
Input : [1 0]
Target : 1
Prediction : 0
Weights : [0. 0.1]
Bias : 0.1

Update 10
Input : [1 1]
Target : 0
Prediction : 1
Weights : [-0.1 0.]
Bias : 0.0

an soooo.. On

Analysis of XOR Non-Convergence

The weights keep oscillating between three states because every correction that fixes one misclassified point breaks another. This happens because XOR is not linearly separable - no straight line can separate $(0,0)$ and $(1,1)$ from $(0,1)$ and $(1,0)$. So the perceptron keeps chasing its tail and never converges.